**Day 12: Functions**

**1. Creating Functions Using def**

A **function** is a named block of code you write once and run whenever you need it. You have already used built-in functions like print(), len(), and input(). Today you write your own.

Why functions matter:

- **No repetition:** write the logic once, use it many times.
- **Readability:** check_port(80) is clearer than ten lines of code.
- **Easier fixing:** a bug is fixed in one place.
- **Testing:** you can test small pieces separately.

**Defining and calling**

python

```
def greet():
    print("Welcome to the lab")


greet()      # call the function
greet()      # call it again
```

Output:

```
Welcome to the lab
Welcome to the lab
```

**Anatomy:**

```
def function_name():
    indented body
```

- def is the keyword that starts a definition.
- The name follows the same rules as variables (use snake_case).
- Parentheses () are required, and the line ends with a colon.
- The body is indented 4 spaces (Day 7 rules).

**Defining does not run the code.** The body only runs when you **call** the function with name().

python

```
def say_hi():
    print("Hi")
```

```
# Nothing printed yet
say_hi()    # now it prints
```

**Define before you call:**

python

```
hello()         # NameError: name 'hello' is not defined


def hello():
    print("Hello")
```

**Docstrings** describe what a function does. Put a triple-quoted string as the first line of the body:

python

```
def show_banner():
    """Print the program banner."""
    print("=" * 30)
    print("  Python for Cybersecurity")
    print("=" * 30)


show_banner()
help(show_banner)      # displays the docstring
```

**Naming tips:** use verbs that say what the function does (check_port, count_failures, is_valid_ip). Avoid names that clash with built-ins, such as print, input, or sum.

**2. Parameters and Arguments**

Functions become powerful when you give them data.

- **Parameter:** the variable name in the definition.
- **Argument:** the actual value you pass when calling.

python

```
def greet(name):              # name is a parameter
    print(f"Hello, {name}")


greet("Alex")                 # "Alex" is an argument
greet("Sam")
```

**Multiple parameters**

python

```
def describe_port(port, service):
    print(f"Port {port} is used by {service}")


describe_port(22, "SSH")
describe_port(443, "HTTPS")
```

By default, arguments match parameters **by position**. The order matters:

python

```
describe_port("SSH", 22)       # runs, but prints "Port SSH is used by 22"
```

**Keyword arguments**

Name the parameter in the call, and order no longer matters:

python

```
describe_port(service="SSH", port=22)
```

**Default values**

A parameter can have a default, used when the caller leaves it out:

python

```
def connect(host, port=22):
    print(f"Connecting to {host}:{port}")


connect("192.168.56.101")          # uses port 22
connect("192.168.56.101", 2222)    # overrides the default
```

**Rule:** parameters with defaults must come **after** those without.

python

```
# def connect(port=22, host):     # SyntaxError
```

**Wrong number of arguments**

python

```
greet()                  # TypeError: missing 1 required positional argument
greet("Alex", "Sam")       # TypeError: takes 1 positional argument but 2 were given
```

Read these messages carefully; they tell you exactly what is wrong.

**Passing lists and other values**

Any type can be an argument:

python

```
def show_ports(ports):
    for p in ports:
        print(f"Port {p}")

show_ports([22, 80, 443])
```

**A caution about mutable arguments:** a function receives the *same* list, not a copy (Day 10's b = a trap again). Changing it inside changes it outside.

python

```
def add_port(ports):
    ports.append(8080)

my_ports = [22, 80]
add_port(my_ports)
print(my_ports)          # [22, 80, 8080]
```

**Never use a mutable default value:**

python

```
def add_item(item, items=[]):       # BAD: the list is shared between calls
    items.append(item)
    return items

def add_item(item, items=None):     # GOOD
    if items is None:
        items = []
    items.append(item)
    return items
```

**Flexible arguments (bonus)**

python

```
def total(*numbers):                # any number of positional arguments
    return sum(numbers)

print(total(1, 2, 3))               # 6
```

```
def show_info(**details):           # any number of keyword arguments
    for key, value in details.items():
        print(f"{key}: {value}")


show_info(name="web-01", port=80)
```

**3. Return Values**

print() **shows** a value on the screen. return **sends a value back** to the caller so the program can use it.

```
python
def add(a, b):
    return a + b


result = add(3, 4)
print(result)          # 7
print(add(10, 5) * 2)    # 30
```

**Print vs. return:**

```
python
def add_print(a, b):
    print(a + b)        # shows it, but gives back None


def add_return(a, b):
    return a + b        # gives the value back


x = add_print(2, 3)      # prints 5
print(x)                # None


y = add_return(2, 3)     # prints nothing
print(y)                # 5
```

Use return when you need the result later; use print only to display.

**return ends the function immediately.** Code after it never runs:

```
python
```

```
def check(n):
    return n * 2
    print("never reached")
```

**Returning booleans**

Functions that answer yes/no questions are very common. Name them with is_ or has_:

python

```
def is_valid_port(port):
    return 0 <= port <= 65535


print(is_valid_port(443))      # True
print(is_valid_port(70000))    # False


if is_valid_port(8080):
    print("OK")
```

**Multiple return statements**

python

```
def classify_port(port):
    if port <= 1023:
        return "Well-known"
    elif port <= 49151:
        return "Registered"
    return "Dynamic/private"


print(classify_port(22))       # Well-known
print(classify_port(50000))    # Dynamic/private
```

**Returning several values**

Python packs them into a tuple, and you unpack them (Day 10):

python

```
def analyze(ports):
    return min(ports), max(ports), len(ports)
```

```
low, high, count = analyze([22, 80, 443])
print(low, high, count)        # 22 443 3
```

**No return means None**

A function without return gives back None, which is Python's "nothing" value.

**Variable scope**

Variables created **inside** a function are **local**: they exist only while the function runs.

python

```
def make_label():
    label = "Server"
    return label

make_label()
print(label)          # NameError: label is not defined
```

Variables defined outside are **global** and can be *read* inside functions, but assigning to them inside creates a new local variable instead. Best practice: pass data in with parameters and get results out with return, rather than relying on globals.

python

```
THRESHOLD = 3          # constants are fine to read in functions

def is_alert(count):
    return count >= THRESHOLD
```

**4. Reusing Functions**

The real payoff: build small functions, then combine them.

**Calling a function from another function**

python

```
def clean(text):
    return text.strip().lower()

def is_yes(answer):
    return clean(answer) in ("y", "yes")

print(is_yes("  YES "))        # True
```

**Reuse with loops**

python

```
def check_port(port):
    return port in (22, 80, 443)


for p in [21, 22, 80, 3389]:
    print(p, check_port(p))
```

**Reuse across files with modules**

Put functions in a file such as helpers.py, then import them from another file in the same folder:

python

```
# helpers.py
def clean(text):
    return text.strip().lower()
```

python

```
# main.py
import helpers
from helpers import clean


print(helpers.clean("  ADMIN "))
print(clean("  Guest "))
```

Every library you will use (socket, hashlib, requests) is just a collection of reusable functions someone else wrote.

**The main() pattern**

Professional scripts keep the program flow in a main() function:

python

```
def main():
    print("Program running")


if __name__ == "__main__":
    main()
```

The if __name__ == "__main__": line means “run main() only when this file is run directly, not when it’s imported.” Treat it as a pattern to copy for now.

**Putting It All Together: Password Checker as Functions**

This rebuilds the Day 9 checker as reusable pieces. It checks **a test password you type** and nothing else.

Create day12_program.py:

python

```
# Program: Day 12 functions
# Purpose: Password strength checker built from reusable functions (practice only)

import string

COMMON_PASSWORDS = ("password", "123456", "12345678", "qwerty",
                    "letmein", "admin", "welcome")


def has_upper(text):
    """Return True if text contains an uppercase letter."""
    return any(ch.isupper() for ch in text)


def has_lower(text):
    """Return True if text contains a lowercase letter."""
    return any(ch.islower() for ch in text)


def has_digit(text):
    """Return True if text contains a digit."""
    return any(ch.isdigit() for ch in text)


def has_special(text):
    """Return True if text contains a punctuation character."""
    return any(ch in string.punctuation for ch in text)
```

```
def is_common(text):
    """Return True if text is a very common password."""
    return text.lower() in COMMON_PASSWORDS



def calculate_score(password):
    """Return a score from 0 to 6."""
    score = 0
    if len(password) >= 8:
        score += 1
    if len(password) >= 12:
        score += 1
    if has_upper(password):
        score += 1
    if has_lower(password):
        score += 1
    if has_digit(password):
        score += 1
    if has_special(password):
        score += 1
    return score



def rate_password(password):
    """Return a rating string for the password."""
    score = calculate_score(password)

    if is_common(password):
        return "Very Weak (common password)"
```

```
    if len(password) < 8:
        return "Weak (too short)"
    if score <= 3:
        return "Weak"
    if score == 4:
        return "Medium"
    if score == 5:
        return "Strong"
    return "Very Strong"


def get_tips(password):
    """Return a list of suggestions."""
    tips = []
    if len(password) < 12:
        tips.append("Use 12 or more characters")
    if not has_upper(password):
        tips.append("Add an uppercase letter")
    if not has_lower(password):
        tips.append("Add a lowercase letter")
    if not has_digit(password):
        tips.append("Add a digit")
    if not has_special(password):
        tips.append("Add a special character")
    if is_common(password):
        tips.append("Avoid common passwords")
    return tips


def show_report(password):
    """Print the full report for a password."""
```

```
    print()
    print(f"Length:     {len(password)}")
    print(f"Score:      {calculate_score(password)} / 6")
    print(f"Result:     {rate_password(password)}")

    tips = get_tips(password)
    if tips:
        print()
        print("Suggestions:")
        for tip in tips:
            print(f"- {tip}")


def main():
    print("=" * 36)
    print("  PASSWORD STRENGTH CHECKER")
    print("=" * 36)
    print("Use made-up test passwords. Type 'quit' to exit.")

    while True:
        password = input("\nTest password: ")
        if password.lower() == "quit":
            print("Goodbye")
            break
        show_report(password)


if __name__ == "__main__":
    main()
```

Sample run:

```
==================================
```

PASSWORD STRENGTH CHECKER

==========

Use made-up test passwords. Type 'quit' to exit.

Test password: Lab2026key

Length: 10

Score: 4 / 6

Result: Medium

Suggestions:

- Use 12 or more characters

- Add a special character

Test password: quit

Goodbye

**Practice Tasks**

1. Write say_hello(name) that prints a greeting, and call it for three names.
2. Write square(n) that returns n * n, then print square(4) and square(square(2)).
3. Write is_even(n) that returns True or False, and use it in a loop over 1 to 10.
4. Write is_valid_port(port) (0 to 65535) and use it to validate user input from Day 7.
5. Write connect(host, port=22) that prints a message, and call it with and without the port.